

1. Importación de librerías

Se cargan las librerías utilizadas para manipulación de datos, cálculo numérico, gráficos y manejo de archivos.

Código ejecutado

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
```

2. Carga de la base de datos

Se importa el archivo Amazon.xlsx mediante pandas y se revisan las primeras observaciones.

Código ejecutado

```
data = pd.read_excel(r"C:\Users\cebal\OneDrive\Documentos\Grandes Bases de Datos I\Amazon.xlsx")
data.head()
```

Resultado

	Unnamed: 0	Velocidad Entrega	Precio	Durabilidad	Imagen Producto	\
0	Adam	205	3	345	235	
1	Anna	9	15	315	33	
2	Bernard	17	26	285	3	
3	Edward	135	5	355	295	
4	Emilia	3	45	48	39	

	Valor Educativo	Servicio Retorno	Tamano Paquete	Calidad Producto	\
0	24	23	26	21	
1	25	4	42	215	
2	43	27	41	26	
3	18	23	39	195	
4	34	46	225	34	

	Numero Estrellas
0	17
1	28
2	33
3	17
4	43

3. Exploración de la base

Se consulta el tamaño de la base y los nombres de sus variables.

Código ejecutado

```
# mostrar el tamaño de la base
data.shape
data.columns
```

Resultado

```
Index(['Unnamed: 0', 'Velocidad Entrega', 'Precio', 'Durabilidad',
      'Imagen Producto', 'Valor Educativo', 'Servicio Retorno',
      'Tamano Paquete', 'Calidad Producto', 'Numero Estrellas'],
      dtype='str')
```

4. Separación de nombres y variables

La primera columna se conserva como identificador de clientes y las demás columnas se utilizan como variables para el clustering.

Código ejecutado

```
# Guardamos a los clientes en una variable
nombres = data.iloc[:, 0]
# Eliminamos la columna de nombres para quedarnos solo con las variables
datos = data.iloc[:, 1:]
```

```
datos.head()
```

Resultado

	Velocidad Entrega	Precio	Durabilidad	Imagen Producto	Valor Educativo	\
0	205	3	345	235	24	
1	9	15	315	33	25	
2	17	26	285	3	43	
3	135	5	355	295	18	
4	3	45	48	39	34	

	Servicio Retorno	Tamaño Paquete	Calidad Producto	Número Estrellas
0	23	26	21	17
1	4	42	215	28
2	27	41	26	33
3	23	39	195	17
4	46	225	34	43

5. Normalización de los datos

Se normalizan las variables antes de aplicar el agrupamiento.

Código ejecutado

```
from sklearn.preprocessing import normalize
# Normalizamos los datos
data_scaled = normalize(datos)
data_scaled
```

Resultado

```
array([[0.43826336, 0.00641361, 0.73756517, 0.50239946, 0.05130888,
        0.04917101, 0.05558462, 0.04489527, 0.03634379],
       [0.02323527, 0.03872544, 0.81323434, 0.08519598, 0.06454241,
        0.01032679, 0.10843125, 0.55506471, 0.0722875 ],
       [0.05723452, 0.08753514, 0.95951982, 0.01010021, 0.14476966,
        0.09090188, 0.13803618, 0.08753514, 0.11110229],
       [0.25885639, 0.00958727, 0.68069644, 0.56564916, 0.03451419,
        0.04410146, 0.07478074, 0.37390368, 0.03259673],
       [0.01197502, 0.17962533, 0.19160036, 0.15567529, 0.13571692,
        0.18361701, 0.89812667, 0.13571692, 0.17164198],
       [0.14494006, 0.251738 , 0.60264551, 0.03661644, 0.03966781,
        0.02898801, 0.73995715, 0.03356507, 0.00457705],
       [0.04755854, 0.02481315, 0.98218723, 0.06823617, 0.07237169,
        0.09304932, 0.07857498, 0.05996511, 0.06410064],
       [0.13323393, 0.04304481, 0.06354234, 0.52268696, 0.05739308,
        0.04509456, 0.70716471, 0.44069685, 0.05944283],
       [0.73804829, 0.0214705 , 0.12613916, 0.63069581, 0.09393342,
        0.00805144, 0.10198485, 0.07246292, 0.12882297],
       [0.00348086, 0.30457545, 0.56564012, 0.00522129, 0.06439595,
        0.0556938 , 0.75708755, 0.04699164, 0.06787681],
       [0.05320841, 0.03547227, 0.19509749, 0.10641681, 0.00886807,
        0.12415295, 0.12858698, 0.9533173 , 0.07537858],
       [0.30081705, 0.01696917, 0.70190646, 0.03548099, 0.00462795,
        0.03856629, 0.6402004 , 0.03856629, 0.04936485],
       [0.03421592, 0.01710796, 0.98981777, 0.0464359 , 0.05132388,
        0.03421592, 0.08065182, 0.05376788, 0.05865587],
       [0.37889397, 0.15360566, 0.08806725, 0.58370152, 0.05529804,
        0.07577879, 0.68610529, 0.05120189, 0.04710574],
       [0.33000964, 0.09127926, 0.69512668, 0.47043927, 0.00421289,
        0.0365117 , 0.04774608, 0.41426742, 0.05476756],
       [0.02878814, 0.00169342, 0.8213087 , 0.3979537 , 0.04572234,
        0.02878814, 0.04064208, 0.3979537 , 0.05757628],
       [0.03645394, 0.46706613, 0.64933583, 0.58098469, 0.08202137,
        0.06607277, 0.07062951, 0.05012417, 0.05240254],
       [0.41749646, 0.0153366 , 0.65606587, 0.36637445, 0.05793828,
        0.02556101, 0.50269982, 0.04771388, 0.04260168],
       [0.3782489 , 0.00999148, 0.69226685, 0.43534307, 0.04710269,
        0.05566682, 0.04853005, 0.42106953, 0.05566682],
       [0.36323349, 0.10046884, 0.76510885, 0.51780094, 0.00463702,
        0.04018754, 0.05255293, 0.00463702, 0.0061827 ],
       [0.386348 , 0.10536764, 0.10068463, 0.00468301, 0.04917156,
        0.04214705, 0.73757346, 0.52683818, 0.06087908],
       [0.03653752, 0.00429853, 0.89194526, 0.26865821, 0.02579119,
        0.03653752, 0.05588091, 0.35462884, 0.00429853],
       [0.02579636, 0.00343951, 0.78248953, 0.61051381, 0.0601915 ,
        0.05847175, 0.0722298 , 0.04471369, 0.00687903],
       [0.03042923, 0.19018267, 0.84948258, 0.06085845, 0.04817961,
        0.06339422, 0.09128768, 0.46911724, 0.05325115],
       [0.45206655, 0.01240967, 0.77117236, 0.04254744, 0.05850273,
```

```

0.04609306, 0.03368339, 0.43433845, 0.06027554],
[0.04106056, 0.18745039, 0.70517053, 0.05177201, 0.06069822,
0.04998677, 0.41953184, 0.52664635, 0.06069822],
[0.05696103, 0.35600642, 0.15664282, 0.11392205, 0.09018829,
0.11866881, 0.17088308, 0.87814917, 0.0996818 ],
[0.03874633, 0.09686582, 0.72276808, 0.45452426, 0.04768779,
0.05811949, 0.49923156, 0.04321706, 0.05811949],
[0.31413959, 0.02513117, 0.88856626, 0.31413959, 0.05564758,
0.03051642, 0.04846725, 0.04846725, 0.06103283],
[0.36927647, 0.3332495 , 0.53139785, 0.49537087, 0.0702526 ,
0.00540405, 0.07565664, 0.4593439 , 0.05584181],
[0.04750309, 0.05066997, 0.00950062, 0.83922133, 0.09817306,
0.00950062, 0.01266749, 0.52253404, 0.08867244],
[0.02461563, 0.03340693, 0.78242552, 0.60659956, 0.05802257,
0.05626431, 0.07208864, 0.04395649, 0.06681386],
[0.04103525, 0.00157828, 0.73389965, 0.46559225, 0.05839632,
0.03787869, 0.03630041, 0.48137504, 0.07102255],
[0.04861252, 0.52901858, 0.09150592, 0.81497458, 0.1000846 ,
0.09722504, 0.12010152, 0.05433164, 0.09150592],
[0.02738021, 0.01140842, 0.87844842, 0.03878863, 0.03878863,
0.02509853, 0.07073221, 0.46774526, 0.00456337],
[0.01909709, 0.35011336, 0.79571219, 0.47742731, 0.05304748,
0.05092558, 0.08063217, 0.03819419, 0.05092558],
[0.03823337, 0.00424815, 0.0615982 , 0.0615982 , 0.07859081,
0.0531019 , 0.98769534, 0.05097782, 0.0615982 ],
[0.00312254, 0.07025712, 0.71037754, 0.04215427, 0.03747046,
0.040593 , 0.5698633 , 0.39812367, 0.04839935],
[0. , 0.17312925, 0.56885325, 0.04451895, 0.01813735,
0.0428701 , 0.73373825, 0.32152575, 0.0230839 ],
[0.0418933 , 0.00349111, 0.11171547, 0.78549943, 0.07331328,
0.07680439, 0.15360878, 0.57603291, 0.04538441],
[0.21476784, 0.03843214, 0.08590714, 0.05199642, 0.05877857,
0.05651785, 0.87037493, 0.41823211, 0.05651785],
[0.57760365, 0.08810903, 0.09398297, 0.07636116, 0.06657127,
0.09006701, 0.44054515, 0.65592278, 0.08419307],
[0.37318997, 0.1751708 , 0.70829934, 0.34272549, 0.05483608,
0.01980192, 0.04721996, 0.44935119, 0.05788253],
[0.07929518, 0.20616748, 0.13638772, 0.74537474, 0.09832603,
0.07929518, 0.58678437, 0.07612338, 0.10466964],
[0.00176146, 0.022899 , 0.57247507, 0.32587042, 0.04227508,
0.02994485, 0.74862124, 0.02818339, 0.04051362],
[0.07177865, 0.35889325, 0.13494386, 0.06603636, 0.1062324 ,
0.04019604, 0.90441099, 0.00861344, 0.11197469],
[0.40592889, 0.24879512, 0.01309448, 0.58925161, 0.0680913 ,
0.08380467, 0.04975902, 0.64162953, 0.01047558],
[0.03079523, 0.35323942, 0.05072156, 0.05072156, 0.06521343,
0.04166414, 0.82422532, 0.42569879, 0.05072156],

```

... [salida recortada para presentación]

6. Conversión a DataFrame

Los datos normalizados se convierten nuevamente en un DataFrame conservando los nombres de las variables.

Código ejecutado

```

# Convertimos los datos normalizados nuevamente en un DataFrame
data_scaled = pd.DataFrame(data_scaled, columns=datos.columns)
data_scaled.head()

```

Resultado

	Velocidad Entrega	Precio	Durabilidad	Imagen Producto	Valor Educativo	\
0	0.438263	0.006414	0.737565	0.502399	0.051309	
1	0.023235	0.038725	0.813234	0.085196	0.064542	
2	0.057235	0.087535	0.959520	0.010100	0.144770	
3	0.258856	0.009587	0.680696	0.565649	0.034514	
4	0.011975	0.179625	0.191600	0.155675	0.135717	

	Servicio Retorno	Tamaño Paquete	Calidad Producto	Numero Estrellas
0	0.049171	0.055585	0.044895	0.036344
1	0.010327	0.108431	0.555065	0.072287
2	0.090902	0.138036	0.087535	0.111102
3	0.044101	0.074781	0.373904	0.032597
4	0.183617	0.898127	0.135717	0.171642

7. Dendrograma inicial

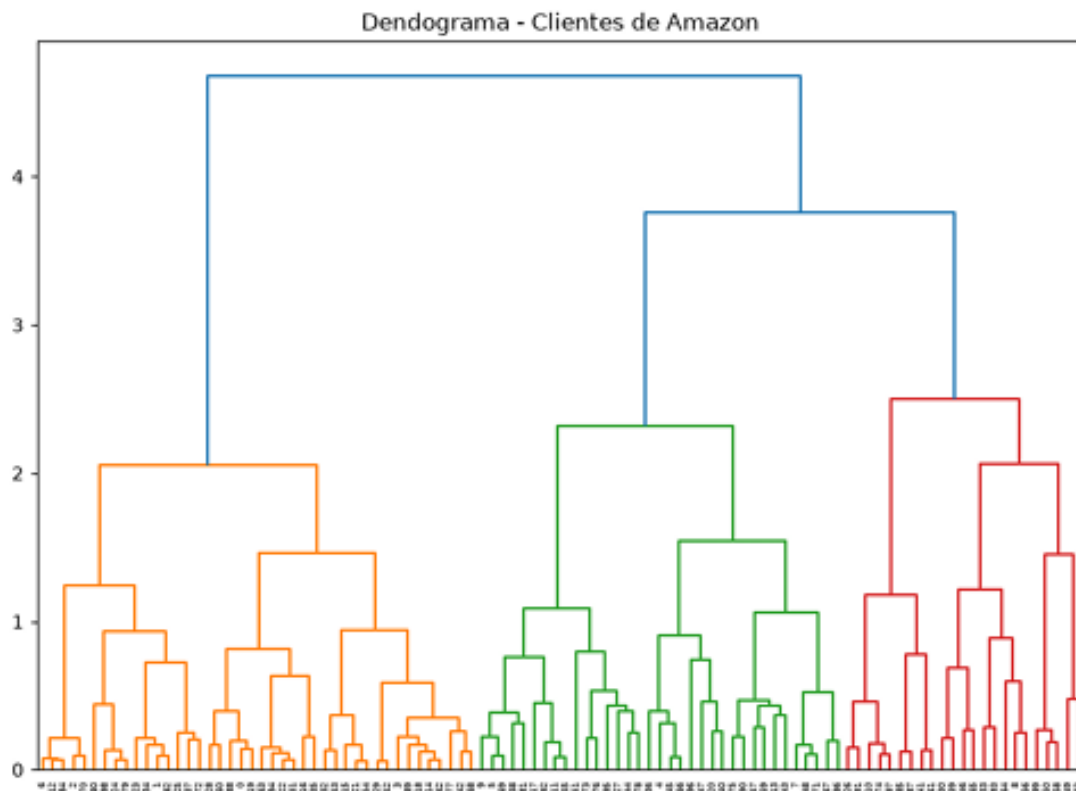
Se construye un dendrograma mediante linkage con método de Ward para observar la estructura jerárquica de los clientes.

Código ejecutado

```
# creamos el dendrograma
import scipy.cluster.hierarchy as shc

plt.figure(figsize=(10,7))
plt.title('Dendrograma - Clientes de Amazon')
dend = shc.dendrogram(
    shc.linkage(data_scaled, method='ward')
)
```

Resultado



8. Número de clusters

A partir del dendrograma se determina trabajar con 4 clusters.

Código ejecutado

```
colores_unicos = set(dend['color_list'])

# Calculamos el número de clusters
num_clusters_optimo = len(colores_unicos)
num_clusters_optimo

# por lo que se van a trabajar con 4 clusters
```

Resultado

4

9. Dendrograma con punto de corte

Se agrega una línea horizontal para visualizar el corte utilizado para separar los grupos.

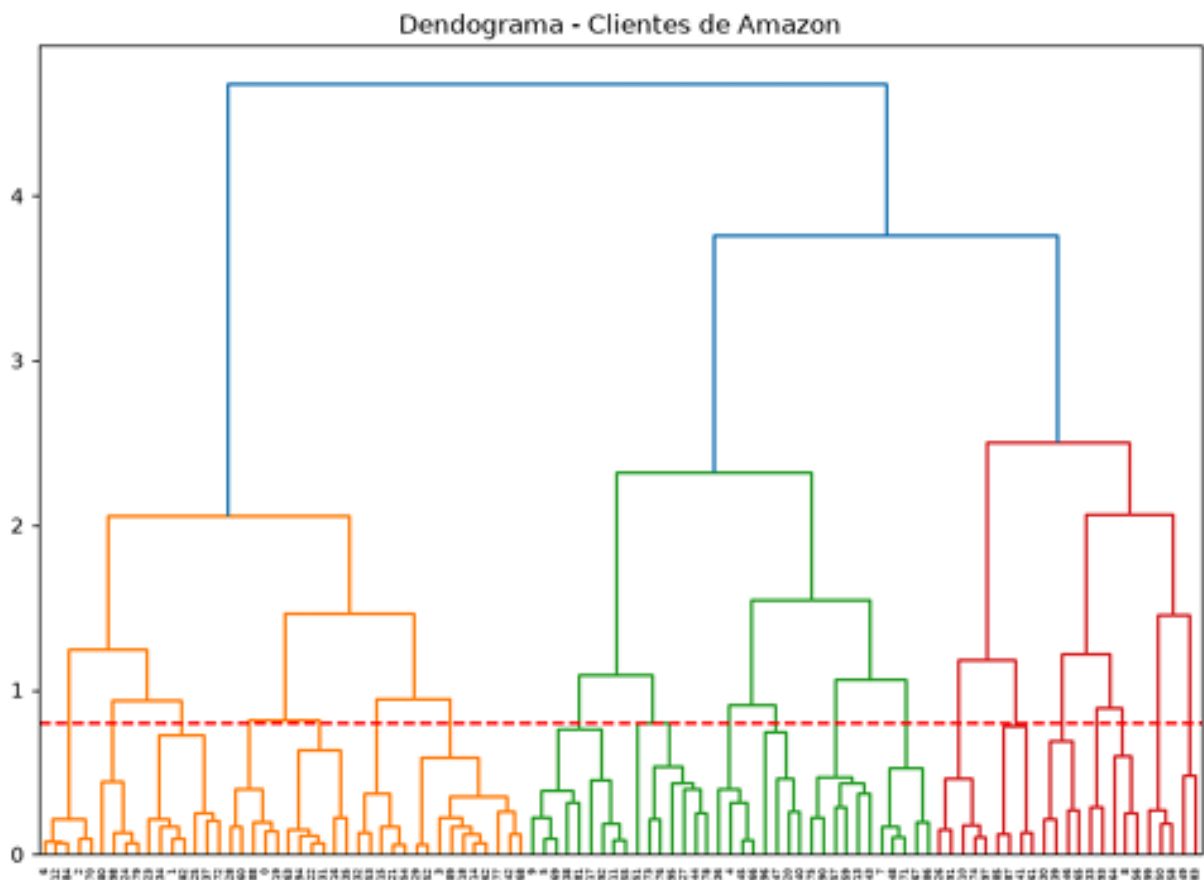
Código ejecutado

```
plt.figure(figsize=(10,7))

plt.title('Dendrograma - Clientes de Amazon')
dend = shc.dendrogram(
    shc.linkage(data_scaled, method='ward')
)

plt.axhline(y=0.8, color='r', linestyle='--')
plt.show()
```

Resultado



10. Clustering jerárquico aglomerativo

Se crea el modelo `AgglomerativeClustering` con 4 grupos y enlace Ward, y se asigna un grupo a cada cliente.

Código ejecutado

```
from sklearn.cluster import AgglomerativeClustering

# Creamos el modelo con 4 grupos
cluster = AgglomerativeClustering(
    n_clusters=4,
    linkage='ward'
)
# Asignamos cada cliente a uno de los 4 grupos
grupos = cluster.fit_predict(data_scaled)
grupos
```

Resultado

```
array([[1, 1, 1, 1, 0, 0, 1, 0, 2, 0, 3, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1,
        1, 1, 1, 1, 3, 0, 1, 1, 2, 1, 1, 2, 1, 1, 0, 1, 0, 2, 0, 3, 1, 0,
        0, 0, 2, 0, 0, 2, 2, 0, 1, 1, 1, 0, 2, 0, 2, 0, 1, 3, 1, 1, 2, 2,
        0, 0, 1, 0, 1, 0, 1, 0, 3, 0, 0, 1, 0, 1, 1, 0, 1, 2, 1, 3, 0, 3,
        1, 1, 0, 3, 0, 2, 1, 0, 0, 3, 1, 2])
```

11. Reducción de dimensión con PCA

Se reducen las variables a dos componentes principales para facilitar la representación gráfica.

Código ejecutado

```
# Análisis gráfico mediante PCA
campos = data_scaled.values
from sklearn import decomposition
pca = decomposition.PCA(n_components=2)

# Ajustamos el modelo PCA
pca.fit(campos)

campos = pca.transform(campos)
campos
```

Resultado

```
array([[ 0.39993219, -0.06035374],
       [ 0.40829583,  0.06402268],
       [ 0.49703182,  0.35955286],
       [ 0.33996675, -0.18434501],
       [-0.58207751,  0.38797653],
       [-0.1723403 ,  0.52962573],
       [ 0.56528441,  0.33770685],
       [-0.5335887 , -0.06994712],
       [-0.09981604, -0.47165743],
       [-0.22195207,  0.56503313],
       [-0.08274005, -0.39712712],
       [ 0.00143465,  0.48936583],
       [ 0.57292039,  0.35927774],
       [-0.51350639, -0.01301312],
       [ 0.35272162, -0.20639728],
       [ 0.4688626 , -0.0168001 ],
       [ 0.25142488, -0.04599019],
       [ 0.05456876,  0.22302796],
       [ 0.36123131, -0.20553368],
       [ 0.41087067, -0.02173183],
       [-0.54207892,  0.03308916],
       [ 0.5138909 ,  0.0900875 ],
       [ 0.4284552 ,  0.0601418 ],
       [ 0.42320562,  0.09945498],
       [ 0.42610215, -0.06318626],
       [ 0.10615885,  0.20376697],
       [-0.19058214, -0.3966612 ],
       [ 0.096838 ,  0.33104612],
       [ 0.51505714,  0.10893109],
       [ 0.17050111, -0.33261099],
       [-0.13793391, -0.63651115],
       [ 0.42206647,  0.05789717],
```

```
[ 0.40326074, -0.12516351],
[-0.21346687, -0.37300285],
[ 0.48923475,  0.13003385],
[ 0.37549596,  0.08899848],
[-0.70102522,  0.46745429],
[ 0.03811566,  0.37986053],
[-0.18646361,  0.43630007],
[-0.14284545, -0.50096372],
[-0.62260912,  0.19528624],
[-0.36295672, -0.28878811],
[ 0.34697878, -0.19244643],
[-0.42217536, -0.03448406],
[-0.16141571,  0.48420739],
[-0.65108758,  0.4197626 ],
[-0.19657428, -0.70259997],
[-0.66940341,  0.17418122],
[-0.50314121, -0.04478847],
[-0.37827002, -0.22768866],
[-0.09257978, -0.29000069],
[-0.15947025,  0.08591885],
[ 0.16414836, -0.33801587],
[ 0.35892963, -0.07853561],
[ 0.49880502,  0.06450716],
[-0.04236318,  0.50393996],
[-0.13801031, -0.49375593],
[-0.57613865,  0.17095284],
[-0.0783886 , -0.35710609],
[-0.67312488,  0.09618175],
[ 0.47969491,  0.06068297],
[-0.39939783, -0.25539679],
[ 0.3695398 , -0.16814344],
[ 0.42508059, -0.03242069],
[-0.02636687, -0.46187498],
[-0.10479979, -0.62527833],
[-0.66976445,  0.42777742],
[-0.55740005, -0.0791324 ],
[ 0.36929226, -0.16505085],
[-0.12646304,  0.50126007],
[ 0.49481837,  0.32229269],
[-0.50365854, -0.01974903],
[ 0.23024785,  0.32414928],
[ 0.02497426,  0.23000985],
[-0.15056994, -0.39659029],
[-0.59247292,  0.24149892],
[ 0.10475686,  0.09868653],
[ 0.45279472, -0.0665796 ],
[-0.2099264 ,  0.34219382],
[ 0.41622673, -0.03730307],
[ 0.14855054,  0.1595983 ],
[-0.11014101,  0.56579396],
[ 0.40616865,  0.06841998],
[-0.23672835, -0.24043862],
[ 0.56568028,  0.36408458],
[-0.23730948, -0.44802811],
[-0.49928987, -0.14100592],
[-0.21649152, -0.46796438],
[ 0.42855949, -0.06683056],
[ 0.31814994, -0.22452555],
[-0.53277044,  0.10971321],
[-0.15466959, -0.45884497],
[-0.11664927,  0.41801606],
[-0.21734196, -0.44145587],
[ 0.45720098,  0.11118217],
[-0.18527671,  0.23439125],
[-0.53527538,  0.14792444],
[-0.19164058, -0.36824856],
[ 0.38487326, -0.03658414],
[ 0.08413049, -0.3546116 ]])
```

12. Visualización de los grupos

Se representan los clientes en el espacio de los dos componentes principales y se colorean según el cluster.

Código ejecutado

```
plt.figure(figsize=(10,7))

# graficamos los clientes utilizando los dos principales componentes
plt.scatter(
    campos[:,0],
```

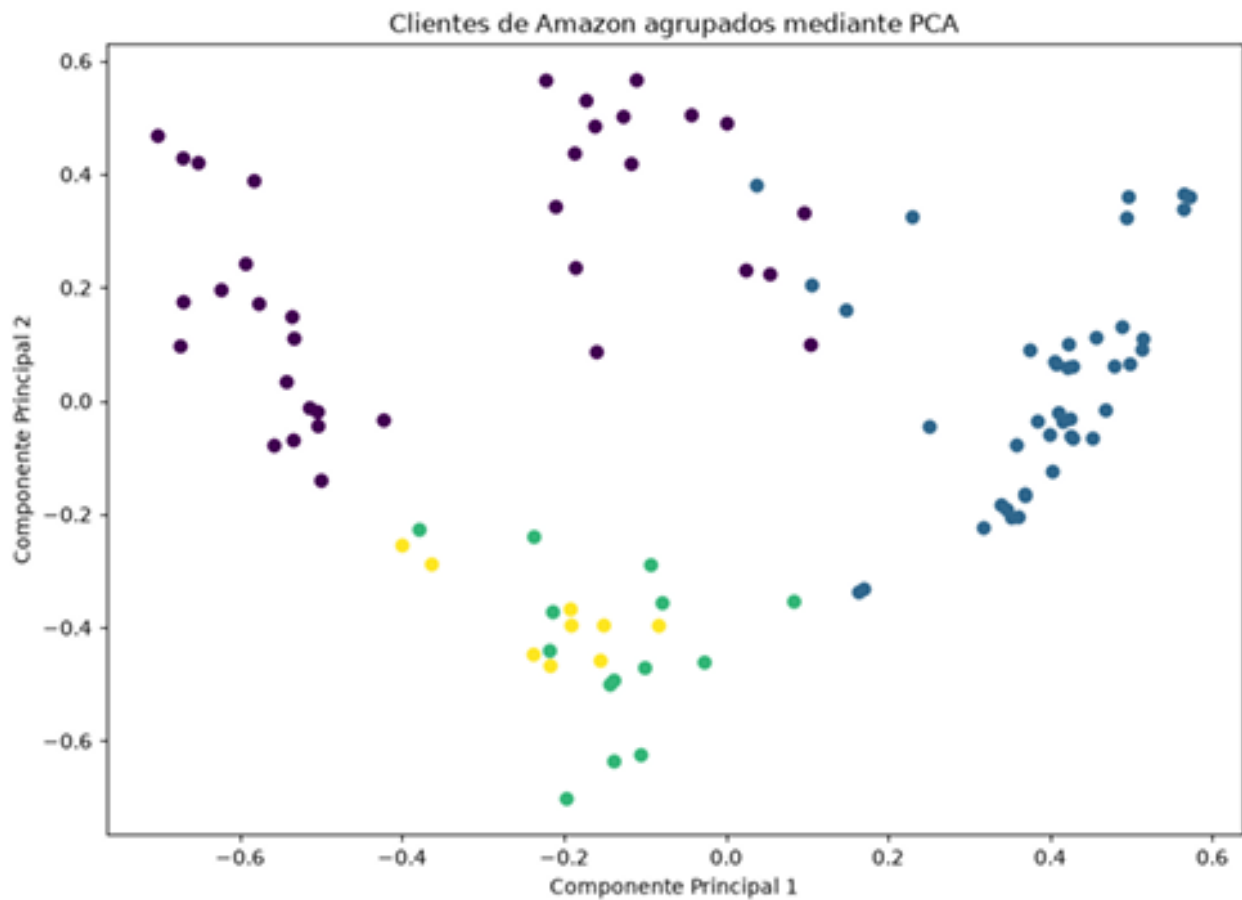
```

campos[:,1],
c=cluster.labels_
)
plt.title("Clientes de Amazon agrupados mediante PCA")

plt.xlabel("Componente Principal 1")
plt.ylabel("Componente Principal 2")
plt.show()

```

Resultado



13. Grupo asignado a cada cliente

Se genera un DataFrame que contiene el grupo correspondiente a cada observación.

Código ejecutado

```
# grupo asignado para cada cliente
dataframe = pd.DataFrame(grupos, columns=['grupo'])
dataframe
```

Resultado

```
   grupo
0      1
1      1
2      1
3      1
4      0
..    ...
95     0
96     0
97     3
98     1
99     2
```

[100 rows x 1 columns]

14. Integración con la base original

Se concatena la asignación de grupos con la base de datos original.

Código ejecutado

```
# Unimos la base original con la columna que obtuvimos del grupo
dataframe2 = pd.concat(
    [data, dataframe],
    axis=1,
    join='inner'
)
dataframe2
```

Resultado

	Unnamed: 0	Velocidad Entrega	Precio	Durabilidad	Imagen Producto	\
0	Adam	205	3	345	235	
1	Anna	9	15	315	33	
2	Bernard	17	26	285	3	
3	Edward	135	5	355	295	
4	Emilia	3	45	48	39	
..	...	...	...	...	...	
95	Teofan	3	8	32	25	
96	Teofil	305	25	46	24	
97	Teofila	1	14	26	25	
98	Teon	155	11	335	34	
99	Teresa	125	9	45	25	

	Valor Educativo	Servicio Retorno	Tamaño Paquete	Calidad Producto	\
0	24	23	26	21	
1	25	4	42	215	
2	43	27	41	26	
3	18	23	39	195	
4	34	46	225	34	
..	...	...	...	...	
95	7	21	42	17	
96	33	28	355	26	
97	24	27	42	185	
98	26	29	42	215	
99	22	3	3	22	

	Numero Estrellas	grupo
0	17	1
1	28	1
2	33	1
3	17	1
4	43	0
..	...	...
95	1	0

```

96          45      0
97          23      3
98          27      1
99          18      2

```

```
[100 rows x 11 columns]
```

15. Identificación por nombre

Se utiliza el nombre de cada cliente como índice para facilitar las consultas individuales.

Código ejecutado

```

dataframe2.index = nombres
dataframe2

```

Resultado

```

          Unnamed: 0  Demanded: 0  Velocidad Entrega  Precio  Durabilidad  \
Adam              Adam              206          3          345
Anna              Anna              9          15          315
Bernard           Bernard           17          26          285
Edward            Edward            135          5          355
Emilia            Emilia            3          45           48
...              ...              ...          ...          ...
Teofan            Teofan            3           8           32
Teofil            Teofil            306         25           46
Teofila           Teofila           1          14           26
Teon              Teon             155         11          335
Teresa            Teresa            125          9           45

          Unnamed: 0  Imagen Producto  Valor Educativo  Servicio Retorno  \
Adam              235              24              23
Anna              33              25              4
Bernard           3              43              27
Edward            295             18              23
Emilia            39              34              46
...              ...              ...              ...
Teofan            25              7              21
Teofil            24              33              28
Teofila           25              24              27
Teon              34              26              29
Teresa            25              22              3

          Unnamed: 0  Tamaño Paquete  Calidad Producto  Numero Estrellas  grupo
Adam              26              21              17          1
Anna              42             215              28          1
Bernard           41              26              33          1
Edward            39             195              17          1
Emilia            225             34              43          0
...              ...              ...              ...          ...
Teofan            42              17              1          0
Teofil            355             26              45          0
Teofila           42             185              23          3
Teon              42             215              27          1
Teresa            3              22              18          2

[100 rows x 11 columns]

```

16. Consulta de Salome

Se identifica el grupo y las características de Salome.

Código ejecutado

```
# consultamos el grupo al que pertenece cada nombre
dataframe2.loc['Salome']
```

Resultado

Unnamed: 0	Salome
Velocidad Entrega	17
Precio	23
Durabilidad	275
Imagen Producto	41
Valor Educativo	4
Servicio Retorno	44
Tamaño Paquete	315
Calidad Producto	28
Número Estrellas	32
grupo	0

Name: Salome, dtype: object

17. Consulta de Stephania

Se identifica el grupo y las características de Stephania.

Código ejecutado

```
dataframe2.loc['Stephania']
```

Resultado

Unnamed: 0	Stephania
Velocidad Entrega	215
Precio	125
Durabilidad	465
Imagen Producto	315
Valor Educativo	34
Servicio Retorno	4
Tamaño Paquete	37
Calidad Producto	306
Número Estrellas	45
grupo	1

Name: Stephania, dtype: object

18. Consulta de Lydia

Se identifica el grupo y las características de Lydia.

Código ejecutado

```
dataframe2.loc['Lydia']
```

Resultado

Unnamed: 0	Lydia
Velocidad Entrega	19
Precio	4
Durabilidad	435
Imagen Producto	145
Valor Educativo	16
Servicio Retorno	21
Tamaño Paquete	28
Calidad Producto	185
Número Estrellas	24
grupo	1

Name: Lydia, dtype: object

19. Clientes del grupo de Salome

Se recuperan todos los clientes pertenecientes al mismo grupo que Salome.

Código ejecutado

```
# Obtenemos el grupo de Salome
grupo_salome = dataframe2.loc['Salome', 'grupo']

# Buscamos todos los clientes que pertenecen al mismo grupo
clientes_salome = dataframe2[dataframe2['grupo'] == grupo_salome]
clientes_salome
```

Resultado

Unnamed: 0		Velocidad Entrega	Precio	Durabilidad	\
Emilia	Emilia	3	45	48	
Fabian	Fabian	95	165	395	
Frank	Frank	65	21	31	
Gabriel	Gabriel	2	175	325	
Henry	Henry	195	11	455	
Isabelle	Isabelle	185	75	43	
Eugenia	Eugenia	245	9	385	
Evdokia	Evdokia	165	45	43	
Florence	Florence	26	65	485	
Jeremiah	Jeremiah	18	2	29	
Joachim	Joachim	0	105	345	
Santiago	Santiago	95	17	38	
Justin	Justin	25	65	43	
Kalya	Kalya	1	13	325	
Larissa	Larissa	25	125	47	
Leon	Leon	17	195	28	
Leonard	Leonard	29	1	44	
Leo	Leo	13	24	41	
Magdalyna	Magdalyna	145	13	385	
Marcel	Marcel	27	125	48	
Maria	Maria	115	225	4	
Maryna	Maryna	21	125	46	
Matthew	Matthew	8	225	32	
Maya	Maya	115	185	415	
Melania	Melania	28	11	41	
Michael	Michael	26	65	455	
Mina	Mina	21	12	47	
Monica	Monica	19	4	415	
Myron	Myron	5	95	355	
Salome	Salome	17	23	275	
Sebastian	Sebastian	12	145	42	
Susanna	Susanna	15	14	39	
Sylvester	Sylvester	155	21	255	
Teofan	Teofan	3	8	32	
Teofil	Teofil	305	25	46	

Unnamed: 0		Imagen Producto	Valor Educativo	Servicio Retorno	\
Emilia		39	34	46	
Fabian		24	26	19	
Frank		255	28	22	
Gabriel		3	37	32	
Henry		23	3	25	
Isabelle		285	27	37	
Eugenia		215	34	15	
Evdokia		2	21	18	
Florence		305	32	39	
Jeremiah		29	37	25	
Joachim		27	11	26	
Santiago		23	26	25	
Justin		235	31	25	
Kalya		185	24	17	
Larissa		23	37	14	
Leon		28	36	23	
Leonard		225	3	24	
Leo		25	36	25	
Magdalyna		35	28	36	
Marcel		275	4	3	
Maria		235	33	22	
Maryna		31	33	39	
Matthew		265	3	25	
Maya		26	3	23	
Melania		155	4	16	

Michael	225	33	27
Mira	245	32	27
Monica	305	22	26
Myron	225	15	31
Salome	41	4	44
Sebastian	295	27	27
Susanna	355	3	38
Sylvester	39	36	4
Teofan	25	7	21
Teofil	24	33	28

	Tamaño Paquete	Calidad Producto	Numero Estrellas	grupo
Unnamed: 0				
Emilia	225	34	43	0
Fabian	485	22	3	0
Frank	345	215	29	0
Gabriel	435	27	39	0
Henry	415	25	32	0
Isabelle	335	25	23	0
Eugenia	295	28	25	0
Evdokia	315	225	26	0
Florence	335	29	39	0
Jeremiah	465	24	29	0
Joachim	445	195	14	0
Santiago	385	185	25	0
Justin	185	24	33	0
Kalya	425	16	23	0
... [salida recortada para presentación]				

20. Clientes del grupo de Stephania

Se recuperan todos los clientes pertenecientes al mismo grupo que Stephania.

Código ejecutado

```
# Obtenemos el grupo de Stephania
grupo_stephania = dataframe2.loc['Stephania', 'grupo']

clientes_stephania = dataframe2[dataframe2['grupo'] == grupo_stephania]
clientes_stephania
```

Resultado

Unnamed: 0	Velocidad Entrega	Precio	Durabilidad	\
Unnamed: 0				
Adam	Adam	205	3	345
Anna	Anna	9	15	315
Bernard	Bernard	17	26	285
Edward	Edward	135	5	355
Philip	Philip	23	12	475
Irene	Irene	14	7	405
Isidore	Isidore	235	65	495
Joseph	Joseph	17	1	485
Eugene	Eugene	16	205	285
Eunice	Eunice	265	7	485
Eva	Eva	235	65	495
Fedir	Fedir	17	2	415
Felix	Felix	15	2	455
Fialka	Fialka	12	75	335
Flavia	Flavia	255	7	435
Flora	Flora	23	105	395
Hannah	Hannah	175	14	495
Helen	Helen	205	185	295
Hilary	Hilary	14	19	445
Lourdes	Lourdes	26	1	465
Ivan	Ivan	12	5	385
Jacob	Jacob	9	165	375
Jervia	Jervia	2	45	455
Judith	Judith	245	115	465
Louise	Louise	225	205	315
Lubomyr	Lubomyr	14	12	335
Lydia	Lydia	19	4	435
Marian	Marian	155	95	495
Markian	Markian	205	55	465
Marko	Marko	15	19	275
Maura	Maura	265	85	425
Maximillian	Maximillian	18	27	295
Methodius	Methodius	18	11	495
Mykyta	Mykyta	165	13	485
Myroslav	Myroslav	225	8	435

Myroslava	Myroslava	275	9	435
Samuel	Samuel	8	14	305
Sarah	Sarah	13	15	425
Stephan	Stephan	145	6	365
Stephania	Stephania	215	125	465
Theodore	Theodore	2	25	335
Teon	Teon	155	11	335

Unnamed: 0	Imagen Producto	Valor Educativo	Servicio Retorno	\
Adam	235	24	23	
Anna	33	25	4	
Bernard	3	43	27	
Edward	295	18	23	
Philip	33	35	45	
Irene	19	21	14	
Isidore	335	3	26	
Joseph	235	27	17	
Eugene	255	36	29	
Eunice	305	33	39	
Eva	335	3	26	
Fedir	125	12	17	
Felix	355	35	34	
Fialka	24	19	25	
Flavia	24	33	26	
Flora	29	34	28	
Hannah	175	31	17	
Helen	275	39	3	
Hilary	345	33	32	
Lourdes	295	37	24	
Ivan	17	17	11	
Jacob	225	25	24	
Jervia	27	24	26	
Judith	225	36	13	
Louise	295	43	34	
Lubomyr	245	25	26	
Lydia	145	16	21	
Marian	225	26	31	
Markian	275	25	27	
Marko	245	34	26	
Maura	185	35	19	
Maximillian	31	45	29	
Methodius	24	29	19	
Mykyta	165	29	15	
Myroslav	23	31	21	
Myroslava	19	36	21	
Samuel	32	23	38	
Sarah	3	28	28	
Stephan	305	2	25	
Stephania	315	34	4	
Theodore	225	22	21	
Teon	34	26	29	

Tamaño Paquete Calidad Producto Numero Estr
... [salida recortada para presentación]

21. Clientes del grupo de Lydia

Se recuperan todos los clientes pertenecientes al mismo grupo que Lydia.

Código ejecutado

```
# Obtenemos el grupo de Lydia
grupo_lydia = dataframe2.loc['Lydia', 'grupo']

# Buscamos todos los clientes del mismo grupo
clientes_lydia = dataframe2[dataframe2['grupo'] == grupo_lydia]
clientes_lydia
```

Resultado

Unnamed: 0	Velocidad Entrega	Precio	Durabilidad	\
Adam	Adam	205	3	345
Anna	Anna	9	15	315
Bernard	Bernard	17	26	285
Edward	Edward	135	5	355
Philip	Philip	23	12	475
Irene	Irene	14	7	405
Isidore	Isidore	235	65	495

Joseph	Joseph	17	1	485
Eugene	Eugene	16	205	285
Eurice	Eurice	265	7	485
Eva	Eva	235	65	495
Fedir	Fedir	17	2	415
Felix	Felix	15	2	455
Fialka	Fialka	12	75	335
Flavia	Flavia	255	7	435
Flora	Flora	23	105	395
Harnuah	Harnuah	175	14	495
Helen	Helen	205	185	295
Hilary	Hilary	14	19	445
Lourdes	Lourdes	26	1	465
Ivan	Ivan	12	5	385
Jacob	Jacob	9	165	375
Jervis	Jervis	2	45	455
Judith	Judith	245	115	465
Louise	Louise	225	205	315
Lubomyr	Lubomyr	14	12	335
Lydia	Lydia	19	4	435
Marian	Marian	155	95	495
Markian	Markian	205	55	465
Marko	Marko	15	19	275
Maura	Maura	265	85	425
Maximillian	Maximillian	18	27	295
Methodius	Methodius	18	11	495
Mykyta	Mykyta	165	13	485
Myroslav	Myroslav	225	8	435
Myroslava	Myroslava	275	9	435
Samuel	Samuel	8	14	305
Sarah	Sarah	13	15	425
Stephan	Stephan	145	6	365
Stephania	Stephania	215	125	465
Theodore	Theodore	2	25	335
Teon	Teon	155	11	335

	Imagen	Producto	Valor	Educativo	Servicio	Retorno	\
Unnamed: 0							
Adam		235		24		23	
Anna		33		25		4	
Bernard		3		43		27	
Edward		295		18		23	
Philip		33		35		45	
Irene		19		21		14	
Isidore		335		3		26	
Joseph		235		27		17	
Eugene		255		36		29	
Eurice		305		33		39	
Eva		335		3		26	
Fedir		125		12		17	
Felix		355		35		34	
Fialka		24		19		25	
Flavia		24		33		26	
Flora		29		34		28	
Harnuah		175		31		17	
Helen		275		39		3	
Hilary		345		33		32	
Lourdes		295		37		24	
Ivan		17		17		11	
Jacob		225		25		24	
Jervis		27		24		26	
Judith		225		36		13	
Louise		295		43		34	
Lubomyr		245		25		26	
Lydia		145		16		21	
Marian		225		26		31	
Markian		275		25		27	
Marko		245		34		26	
Maura		185		35		19	
Maximillian		31		45		29	
Methodius		24		29		19	
Mykyta		165		29		15	
Myroslav		23		31		21	
Myroslava		19		36		21	
Samuel		32		23		38	
Sarah		3		28		28	
Stephan		305		2		25	
Stephania		315		34		4	
Theodore		225		22		21	
Teon		34		26		29	

Tamaño Paquete Calidad Producto Numero Estr
... [salida recortada para presentación]

